DDoS Detection & Mitigation using eBPF and XDP

A Detailed Study and Project Guide (Written for Beginners)

Reproducing and extending the paper "eBPF-Based Real-Time DDoS Mitigation for IoT Edge Devices" (arXiv:2508.00851)

1. Problem Statement and Issues

1.1 What is DDoS?

DDoS stands for Distributed Denial of Service. It is an attack where many computers (called bots) send a huge amount of traffic to one target (a server or device) at the same time. The goal is to overload the target so that it can no longer serve normal, real users. It is like hundreds of people standing in front of a shop door at once, so the real customers cannot get in.

1.2 The two attacks we study

Two kinds of flood attacks are covered in this project:

- **UDP flood —** the attacker sends a very large number of UDP packets (small pieces of data) to the target. The target wastes all its time and memory trying to handle packets that are useless. In our filter we define a UDP flood as: a sender sending more than 800 packets in 1 second.
- **SYN flood —** the attacker pretends to start thousands of new connections at once. This can exhaust the target. In our filter we define a SYN flood as: a sender trying to start more than 200 new connections in 1 second.

1.3 The real problem: traditional defenses are too slow

Normally, a server defends itself using a firewall (like iptables). But there is a big problem:

1. By the time the firewall sees the packet, the kernel (the core of the operating system) has already spent memory and CPU time on it.
2. The kernel has to reserve a block of memory for every single packet that arrives. This is called "allocating an sk_buff".
3. When millions of bad packets arrive, the kernel runs out of CPU and memory, and the server freezes before the firewall can even drop the packets.

In simple words: the firewall is like a security guard standing INSIDE the building. The attackers are able to fill the building with garbage before the guard can stop them.

1.4 The idea / solution

What if we could check every packet at the very front door, BEFORE the kernel spends any memory on it? That is exactly what this project does, using two technologies:

- **eBPF —**
- **XDP —** a safe way to run small custom programs inside the Linux kernel.

eBPF lets us write a tiny program that runs inside the kernel. XDP (eXpress Data Path) is the earliest point on the network card where a packet can be inspected. Together, they let us throw away bad packets at the very front door, before they use up any resources. This is much faster and safer than a firewall.

1.5 The specific goals of this project

4. Reproduce the paper's result: block UDP floods in a simulated network, and measure that ~99% of bad packets are dropped.
5. Extend the paper: also block SYN floods (which the paper did not cover).
6. Combine both protections into one program, so one filter stops both kinds of attacks.
7. Keep normal (legitimate) traffic working — do not block real users.
8. Validate everything with live attacks and measure CPU and response time.

2. What Was Implemented From the Paper (and What Was Added)

2.1 What the paper proposed

The paper (Tolay, 2025) built a system with two parts:

- **Kernel part (eBPF/XDP):**
- **User-space part (controller):** a program attached to the network card that counts packets per sender and drops senders who send too fast.

a normal program that reads alerts from the kernel and blocks the attacker's IP using a firewall, and writes a log.

2.2 What we reproduced from the paper

From the paper	Status in this project	Result
Per-sender UDP packet-rate detection (count packets per IP in 1 second)	Reproduced exactly	99.7% of UDP packets dropped
eBPF/XDP program attached to the victim's network interface	Reproduced (native XDP inside the container)	Works
Kernel + user-space controller hybrid design	Reproduced (controller.py)	Detection to block in < 2 s
Rate-based detection (a sender over a limit is flagged)	Reproduced (800 packets/sec limit)	Works
Docker-based simulation with victim / attacker / benign containers	Reproduced	Works

2.3 What we added or did differently (the extension)

Extension / difference	What it means	Result
SYN flood detection	The paper only covered UDP floods. We added detection of SYN floods (too many new connections per second	99.9% of SYN packets dropped

	from one sender).	
Combined program	The paper used only a UDP filter. We wrote ONE program that detects BOTH UDP and SYN floods at the same time.	99.8% combined drop rate
Separate thresholds per attack	UDP uses an 800 packets/sec limit; SYN uses a 200 connections/sec limit, because the two attacks look different.	Both work together
Legitimate-traffic test	We explicitly tested that normal TCP connections and slow UDP traffic pass through with 0% drops.	0% false drops on normal traffic
Clear explanation of the "limit" trade-off	We documented that normal traffic above the limit would be wrongly blocked (a known weakness).	Documented in results

3. What We Need (Tools and Setup)

3.1 The tools and why we need each one

Tool	What it is	Why we need it
Docker	A tool that runs small separate copies of Linux called "containers".	To build the fake test network with the victim, attacker, and normal user.
clang + llvm	A compiler (turns code into a runnable program).	To turn our C code into eBPF code that the kernel can run.
libbpf-dev	A library of helpers for writing eBPF programs.	Provides ready-made functions and headers for eBPF.
bpftool	A command-line tool for managing eBPF programs.	To load our program into the kernel and attach it to a network card.
hping3	A tool that can send custom or high-speed packets.	To launch the UDP and SYN flood attacks.
iperf3	A tool that sends normal traffic between two computers.	To generate legitimate traffic from the normal user.
Linux kernel (4.x+)	The core of the operating system.	eBPF and XDP are features of a modern Linux kernel.

3.2 The three "computers" in the simulation

Name	IP address	Role
victim	172.30.0.10	The target. It runs a server and has the protective filter.
attacker	172.30.0.20	Launches the UDP and SYN flood attacks.
benign	172.30.0.30	A normal user sending normal (good) traffic.

All three are containers connected by a private "fake switch" (Docker calls it a bridge). They use the private address range 172.30.0.0/24, so the attack never leaves your computer.

3.3 Important safety note

The attack is fully contained inside the private Docker network. It only affects the three fake devices. It does NOT touch your real internet connection, so it is safe to run on your own computer.

4. Detailed Implementation (Step by Step)

This section walks through the whole project from start to finish. We explain what each step does and WHY it is needed.

4.1 Step 1 — Install the tools

We first install everything listed in Section 3.1. This is the "workshop" — we cannot build anything without the right tools.

Why we need each: Docker builds the test network, clang builds the kernel programs, hping3 creates the attacks, iperf3 creates normal traffic, and bpftool loads our code.

4.2 Step 2 — Write the eBPF/XDP filter programs (the heart of the project)

We wrote three small C programs. Each one is a filter that looks at every packet and decides: let it through (PASS) or throw it away (DROP).

File	What it does
udp_filter_kern.c	Stops UDP floods. Reproduces the paper.
syn_filter_kern.c	Stops SYN floods. Our new addition.
ddos_filter_kern.c	Stops BOTH UDP and SYN floods in one program. Our main result.

How the UDP filter works (the paper's design)

9. For each incoming packet, read the sender's IP address from the header.
10. Keep a list (a "map") that remembers how many packets each sender has sent in the last 1 second.
11. Every time a packet arrives, add 1 to that sender's count.
12. If a sender's count goes above 800 in 1 second, start throwing that sender's packets away (XDP_DROP).
13. If the count resets after 1 second, let normal traffic through again.

How the SYN filter works (our extension)

14. For each TCP packet, check a flag called SYN. A packet with SYN set (and no ACK) means "I want to start a new connection".
15. Keep a list that remembers how many new-connection packets each sender has sent in 1 second.

16. If a sender tries to start more than 200 new connections in 1 second, throw that sender's SYN packets away.

Why these two need different limits

A UDP flood is just data being sprayed, so we use a high limit (800 packets/s). A SYN flood is about starting connections, which is a rarer action for normal users, so we use a low limit (200/s). Using the right limit for each attack makes the filter accurate.

Where the filter runs (the key idea: XDP)

The filter is attached to the victim's network card at a special point called XDP. XDP is the VERY FIRST place a packet arrives. At this point the kernel has NOT yet reserved memory for the packet. So if the filter says DROP, the packet is gone instantly — it used up no memory and almost no CPU. This is why eBPF+XDP is so much faster than a normal firewall.

4.3 Step 3 — Build (compile) the programs

We use the command "make". This runs clang, which turns our C code into eBPF code. Before the kernel allows this code to run, it runs a safety check (called the "verifier"). If the code is safe (it must finish quickly and cannot crash the system), the kernel accepts it.

WHY: this safety check is what makes eBPF trustworthy. It lets us run custom code inside the kernel without risking the whole computer.

4.4 Step 4 — Build the fake test network (Docker)

We run a setup script that:

- creates a private network (172.30.0.0/24);
- starts the victim, attacker, and benign containers;
- installs the needed tools inside each container;
- starts a UDP server on the victim (this is the service being protected).

WHY: we need a safe, isolated place to run the attacks and measure the protection. Docker gives us this without touching the real network.

4.5 Step 5 — Attach the filter to the victim

We copy the compiled program into the victim container and attach it to the victim's network card (eth0) at the XDP point. From now on, EVERY packet coming to the victim passes through our filter first.

WHY: attaching at the victim's network card means we protect the victim itself. This matches the paper, which protects the target device.

4.6 Step 6 — Run the user-space controller

The filter cannot easily talk to the outside world, and writing a firewall rule or a log file is slow work. So we use a normal program (controller.py) that:

- continuously reads the kernel's "message board" (trace_pipe);
- when the filter writes "UDP FLOOD DETECTED from <ip>" or "SYN FLOOD DETECTED from <ip>", it reads the attacker's IP;
- adds a firewall rule to permanently block that IP (a second line of defense);
- writes the event to a log file (alerts.log).

WHY: this gives us the best of both worlds — the filter is super fast (in the kernel), and the controller is flexible (blocking IPs, logging, and later alerting). This is the same "hybrid" design as the paper.

4.7 Step 7 — Launch the attacks and measure

We use hping3 to send a UDP flood and a SYN flood at the victim. We then check:

- how many packets the filter saw;
- how many it threw away (the drop rate);
- that normal traffic still gets through;
- how fast the victim responds (ping time).

Expected results (we measured these)

Test	Result
UDP flood drop rate	99.7%
SYN flood drop rate	99.9%
Combined UDP + SYN	99.8%
Normal UDP (slow) preserved	0% dropped
Normal TCP connection preserved	0 dropped
Time from detection to blocking	< 2 seconds

4.8 Step 8 — The optional benchmark (CPU and response time)

We can measure the computer's CPU usage and the victim's response time during a flood, with the filter OFF and then ON, and compare them.

WHY: this shows that the filter protects the victim. On this 16-core computer, the flood does not fully use a core, so the CPU number is similar. On a small device like

a Raspberry Pi, the difference would be much bigger (the paper shows 45% down to 12%).

5. The Complete Working (End to End)

Here is the full journey of a packet during an attack, in simple steps:

17. The attacker container sends a flood of packets to the victim.
18. Each packet arrives at the victim's network card.
19. At the XDP point, our filter checks the packet's sender and counts it.
20. If the sender is over the limit, the filter throws the packet away instantly (XDP_DROP). This uses almost no CPU or memory.
21. If the sender is normal, the packet is allowed through and the server handles it normally.
22. When the filter first catches an attack, it leaves a note on the kernel's message board.
23. The controller reads the note, adds a firewall rule to permanently block the attacker's IP, and saves the event to a log.
24. Result: the attack is stopped at the front door, normal users keep working, and the victim stays fast and available.

6. To-Do List: What You Need to Do to Run the Project

Follow these steps in order. Each one tells you what to run and what you should see. This will reproduce the results that solve the problem statement.

To-Do 1 — Install the tools

Run in a terminal:

Command	Purpose
sudo apt update	Refresh the list of available software.
sudo apt install -y docker.io clang llvm libbpf-dev libelf-dev pahole linux-tools-common linux-tools-generic hping3 iperf3 tcpdump python3-requests	Install everything we need.
sudo usermod -aG docker $USER	Let your username use Docker (then log out/in or run "newgrp docker").

Check: run "docker run --rm hello-world" — you should see "Hello from Docker!".

To-Do 2 — Build the filter programs

Run:

- cd ~/Desktop/CN/ebpf
- make

Expected: no errors, and .o files appear. This means our code compiled into safe eBPF code.

To-Do 3 — Build the fake test network

Run:

- cd ~/Desktop/CN/testbed
- sudo bash setup_testbed.sh

Expected: three containers start (victim, attacker, benign) on 172.30.0.0/24.

To-Do 4 — Attach the combined filter to the victim

Run:

- sudo docker cp ~/Desktop/CN/ebpf/ddos_filter_kern.o victim:/tmp/ddos_filter_kern.o

- sudo docker exec victim bash -c 'BPF=/usr/lib/linux-tools/5.15.0-190-generic/bpftool; rm -rf /sys/fs/bpf/ddos; $BPF prog loadall /tmp/ddos_filter_kern.o /sys/fs/bpf/ddos; ip link set dev eth0 xdp pinned /sys/fs/bpf/ddos/xdp'

Expected: the victim's network card shows the filter attached (look for "xdp" and "xdp_ddos_filter").

To-Do 5 — Start the controller

Run:

- cd ~/Desktop/CN/testbed
- sudo bash start_controller.sh

Expected: "controller started". This helper will block attackers and log events.

To-Do 6 — Launch a UDP flood and verify

Run:

- for i in 1 2 3 4; do sudo docker exec -d attacker hping3 -2 -p 5201 --flood 172.30.0.10; done
- sleep 8
- sudo docker exec attacker pkill -9 hping3

Then read the counters:

- sudo docker exec victim bash -c '/usr/lib/linux-tools/5.15.0-190-generic/bpftool map dump name stats_map'

Expected: about 99.7% of UDP packets dropped, and the controller log shows "UDP FLOOD detected from 172.30.0.20" and "BLOCKED 172.30.0.20".

To-Do 7 — Launch a SYN flood and verify

Run:

- for i in 1 2 3 4; do sudo docker exec -d attacker hping3 -S -p 5201 --flood 172.30.0.10; done
- sleep 8
- sudo docker exec attacker pkill -9 hping3

Expected: about 99.9% of SYN packets dropped, and the controller log shows "SYN FLOOD detected from 172.30.0.20".

To-Do 8 — Verify normal traffic is preserved

Run:

- sudo docker exec benign iperf3 -c 172.30.0.10 -p 5201 -u -b 3M -t 5

Expected: the "udp_dropped" counter does NOT increase — normal traffic passes through with 0% drops.

To-Do 9 — (Optional) Run the CPU / latency benchmark

Run:

- cd ~/Desktop/CN/testbed
- sudo bash bench_run.sh filter-off 10
- sudo bash bench_run.sh filter-on 10

This prints CPU and response time with the filter off and on. Use it to show the protection works.

To-Do 10 — Clean up

Run:

- sudo docker exec victim ip link set dev eth0 xdp off
- sudo pkill -f controller.py
- sudo docker rm -f victim attacker benign
- sudo docker network rm ddos-net

7. Expected Results (to confirm the problem is solved)

If all the to-do steps worked, you should see these results, which prove the problem statement has been solved:

Metric	Expected value	What it proves
UDP flood drop rate	~99.7%	The victim is protected from UDP floods.
SYN flood drop rate	~99.9%	The victim is protected from SYN floods (our extension).
Combined drop rate	~99.8%	One program stops both attacks.
Normal traffic preserved	0% dropped	Real users are not blocked.
Detection to block time	< 2 s	The system responds quickly.
Victim stays responsive	Stable ping	The service remains available.

These numbers match the paper's result (~99% for UDP floods), and extend it by also covering SYN floods.

8. Conclusion

This project showed that using eBPF and XDP, we can detect and stop DDoS attacks (UDP and SYN floods) at the earliest possible point on the network card — before they waste memory or CPU. We reproduced the paper's UDP flood protection (99.7% drop) and extended it with SYN flood protection (99.9% drop), combined into one program (99.8% overall). Normal users keep working, and the system responds in under two seconds. This is a lightweight, fast, in-kernel defense that works even on small devices, solving the problem of traditional firewalls being too slow under attack.